

Visión del Producto: "AutoDoc CLI" (Generador Autónomo de Documentación)

El Problema (Por qué lo construimos)

Escribir documentación técnica es la tarea más odiada por los desarrolladores. Cuando el código cambia, la documentación se pudre. Los equipos pierden semanas intentando entender arquitecturas heredadas, y los pipelines de CI/CD despliegan código nuevo con manuales obsoletos.

La Solución: El Producto Final

AutoDoc CLI es un ejecutable binario nativo de apenas unos megabytes. No requiere instalar Java, ni Node, ni Python. Lo dejas caer en cualquier terminal (Linux, Mac o Windows) o en cualquier pipeline de CI/CD, y actúa como un **Arquitecto de Software en la sombra**.

El "Momento Mágico" (Experiencia de Usuario)

Imagina que terminas un *sprint* masivo. Has refactorizado 15 clases, cambiado la lógica de la base de datos y añadido nuevos endpoints. En lugar de pasar el viernes documentando, abres tu terminal y escribes:

```
$ autodoc generate --workspace ./mi-proyecto --diff origin/main
```

Lo que ocurre en los siguientes 4 segundos:

1.  **Arranque Instantáneo:** Gracias a GraalVM, el CLI arranca en 15 milisegundos.
2.  **Análisis Quirúrgico:** No lee todo a lo bruto. Usa JavaParser para construir un mapa mental (AST) de tu código, podando la lógica interna y quedándose solo con las firmas y dependencias.
3.  **Ahorro Inteligente:** Compara tu rama con main (Git diff). Si solo cambiaste 2 archivos, solo envía esos 2 archivos al LLM (Gemini), inyectando la estructura del resto del proyecto usando **Context Caching** para que el modelo sepa exactamente dónde encajan tus cambios.
4.  **Redacción Autónoma:** En la carpeta /docs aparecen mágicamente nuevos archivos Markdown (User_Service_API.md, Architecture_Flow.md) perfectamente formateados, con diagramas Mermaid sugeridos y explicaciones de por qué se hicieron los cambios.

Bajo el Capó: Ingeniería de Élite

Lo que hace a esta herramienta superior a cualquier plugin de IDE comercial es cómo fue construida (orquestrada en Google Antigravity) y su arquitectura técnica:

- **Zero-Cost Development Mode:** Durante el desarrollo local, si le pasas el flag `--local`, el CLI se enruta automáticamente hacia tu instancia de **OpenClaw** (<http://127.0.0.1:18789>). Puedes documentar cientos de veces al día sin gastar un solo céntimo en tokens.
- **Inmunidad a la "Saturación de Contexto":** Al usar el patrón ModifierVisitor de JavaParser, el modelo jamás se confunde leyendo "ifs" o bucles "for". Solo recibe la estructura pura (interfaces, clases, dependencias). Documenta mejor porque se enfoca solo en el diseño.
- **Seguridad de Grado Militar (SBOM):** Al estar compilado con GraalVM Native Image en GitHub Actions, el binario incluye una Lista de Materiales de Software (SBOM). Sabes exactamente qué contiene, sin vulnerabilidades dinámicas.



El Impacto Final: CI/CD en Piloto Automático

El verdadero poder del producto no está en tu portátil, está en la nube.

El flujo de trabajo en tu empresa pasará a ser este:

1. Un desarrollador hace un *Pull Request*.
2. GitHub Actions detecta el PR y descarga tu **AutoDoc CLI**.
3. El CLI analiza los commits, genera la documentación de la API actualizada y los manuales.
4. El CLI hace un commit automático en una rama separada o comenta en el PR: *"He actualizado la documentación en el directorio /docs basándome en tus cambios. Por favor, aprueba"*.

El Resultado:

Tu código y tu documentación **jamás** volverán a estar desincronizados. Has automatizado el trabajo de un Tech Lead dedicado exclusivamente a mantener los manuales al día.